

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 14
“SORTING”



DISUSUN OLEH:
Andromeda Alike Ramadhani
109082500156
S1 IF-13-02

DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

DASAR TEORI

Pengurutan data atau sorting merupakan salah satu proses fundamental dalam algoritma dan struktur data yang bertujuan untuk menyusun kembali sekumpulan data yang acak ke dalam urutan tertentu yang teratur. Secara umum, terdapat dua jenis arah pengurutan, yaitu urutan menaik (ascending) dari nilai terkecil ke terbesar, dan urutan menurun (descending) dari nilai terbesar ke terkecil. Pengurutan data sangat penting dalam dunia komputasi karena struktur data yang teratur akan meningkatkan efisiensi dan kecepatan dalam proses pencarian data (searching), pencocokan data, hingga pemrosesan data yang lebih kompleks.

Terdapat beberapa metode algoritma sorting yang dapat digunakan, antara lain yaitu sebagai berikut:

1. Selection Sort

Selection Sort adalah salah satu algoritma pengurutan berbasis perbandingan yang mengusung prinsip kerja yang sangat sederhana, yaitu dengan memilih elemen ekstrem (terkecil atau terbesar) dari sekumpulan data acak untuk ditempatkan pada posisi yang tepat. Proses algoritma ini bekerja secara berulang dengan membagi array secara virtual menjadi dua bagian, yaitu bagian yang sudah terurut di sisi kiri dan bagian yang belum terurut di sisi kanan. Pada setiap iterasi di bagian yang belum terurut, algoritma akan memindai elemen satu per satu untuk mencari nilai minimum (untuk urutan ascending). Setelah nilai minimum ditemukan, elemen tersebut akan ditukarkan (swapping) dengan elemen pertama yang berada di bagian belum terurut. Langkah ini terus diulang hingga seluruh elemen berpindah ke bagian yang terurut.

2. Insertion Sort

Insertion Sort merupakan algoritma pengurutan yang mengadopsi analogi cara manusia dalam mengurutkan kartu permainan di tangan. Prinsip utama dari algoritma ini adalah menyisipkan (insert) setiap elemen dari bagian array yang belum terurut ke dalam posisi yang tepat di bagian array yang sudah terurut. Algoritma ini bekerja dengan memisahkan elemen pertama sebagai bagian yang dianggap sudah terurut. Kemudian, elemen berikutnya (elemen kedua hingga terakhir) akan diambil satu per satu sebagai nilai kunci (key). Nilai kunci ini akan dibandingkan secara mundur dengan elemen-elemen yang berada di sebelah kirinya yang sudah terurut. Jika elemen di sebelah kiri lebih besar dari nilai kunci (pada urutan ascending), maka elemen tersebut akan digeser ke kanan untuk memberikan ruang bagi nilai kunci. Proses pergeseran dan perbandingan ini berhenti ketika ditemukan posisi yang tepat di mana nilai kunci tidak lagi lebih kecil dari elemen di kirinya, lalu nilai kunci tersebut disisipkan di posisi tersebut.

CONTOH SOAL

1. Latihan1

Source Code:

```
package main

import "fmt"

type arrInt [4321]int

func main() {
    var i int = 1
    var n, idx_min, j, t int
    var a arrInt

    fmt.Scan(&n)

    for i = 0; i < n; i++ {
        fmt.Scan(&a[i])
    }

    i = 1
    for i <= n-1 {
        idx_min = i - 1
        j = i

        for j < n {
            if a[idx_min] > a[j] {
                idx_min = j
            }
            j = j + 1
        }
    }
```

```

        t = a[idx_min]

        a[idx_min] = a[i-1]

        a[i-1] = t

        i = i + 1

    }

    for i = 0; i < n; i++ {

        fmt.Print(a[i], " ")

    }

}

```

The screenshot shows a Go IDE with a code editor on the left and an output window on the right. The code in the editor is a Go program that reads an array of integers and prints them. The output window shows the result of running the program: 109082500156, IF-13-02, and Andromeda Alika Ramadhani.

```

ContohSoal1 > guided1.go > main
1 package main
2
3 import "fmt"
4
5 type arrInt [4321]int
6
7 func main() {
8     var i int = 1
9     var n, idx_min, j, t int
10    var a arrInt
11
12    fmt.Scan(&n)
13
14    for i = 0; i < n; i++ {
15        fmt.Scan(&a[i])
16    }
17
18    i = 1
19    for i <= n-1 {
20        idx_min = i - 1
21        j = i
22
23        for j < n {
24            if a[idx_min] > a[j] {
25                idx_min = j
26            }
27            j = j + 1
28        }
29    }
30
31    for i = 0; i < n; i++ {
32        fmt.Print(a[i], " ")
33    }
34
35    fmt.Println()
36 }

```

Output:

```

109082500156
IF-13-02
Andromeda Alika Ramadhani

```

Output:

The screenshot shows a terminal window with the command prompt PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14>. The command go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14\ContohSoal1\guided1.go" is executed. The output is 5, 7 2 9 1 4, 1 2 4 7 9, and PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14>.

```

PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14\ContohSoal1\guided1.go"
5
7 2 9 1 4
1 2 4 7 9
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14>

```

Deskripsi Program:

Program diatas digunakan untuk mengurutkan sekumpulan data bilangan menggunakan algoritma selection sort ascending. Pada bagian awal program, didefinisikan tipe data arrInt, yaitu array bertipe integer dengan kapasitas maksimum sebanyak 4321 elemen. Array ini digunakan sebagai tempat penyimpanan data angka yang akan diproses oleh program. Selain itu, program juga mengimpor package fmt yang digunakan untuk menerima input dari pengguna dan menampilkan output ke layar.

Pada fungsi main, program dimulai dengan mendeklarasikan beberapa variabel, seperti n untuk menyimpan jumlah data yang akan dimasukkan, idx_min untuk menyimpan indeks nilai terkecil, j sebagai variabel pembanding dalam proses pencarian nilai minimum, t sebagai variabel sementara saat pertukaran data dilakukan, dan i sebagai variabel perulangan. Selanjutnya, pengguna diminta memasukkan jumlah data (n) yang akan diurutkan, kemudian program melakukan perulangan untuk membaca setiap elemen array dan menyimpannya ke dalam variabel a.

Setelah proses input selesai, program menjalankan algoritma selection sort untuk mengurutkan data. Proses pengurutan dilakukan melalui perulangan utama menggunakan variabel i, yang dimulai dari indeks pertama hingga indeks terakhir sebelum elemen akhir. Pada setiap iterasi, program menganggap elemen saat ini sebagai nilai terkecil sementara dengan menyimpan indeksnya ke dalam variabel idx_min. Kemudian dilakukan perulangan kedua menggunakan variabel j untuk mencari elemen dengan nilai lebih kecil pada bagian array yang belum terurut. Jika ditemukan elemen yang lebih kecil, maka indeks nilai minimum diperbarui.

Setelah nilai terkecil ditemukan, program melakukan proses pertukaran (swap) antara elemen pada posisi saat ini dengan elemen bernilai minimum menggunakan variabel sementara t. Langkah ini dilakukan secara berulang hingga seluruh elemen tersusun dari nilai terkecil ke terbesar. Metode selection sort bekerja dengan cara memilih elemen minimum pada setiap tahap pengurutan dan menempatkannya pada posisi yang sesuai.

Pada bagian akhir program, dilakukan perulangan untuk menampilkan seluruh data yang telah diurutkan. Setiap elemen dicetak ke layar dan dipisahkan dengan spasi agar hasil output lebih mudah dibaca.

2. Latihan2**Source Code:**

```
package main

import "fmt"

func main() {
    arr := []int{5, 2, 9, 1, 5, 6}

    for i := 1; i < len(arr); i++ {
```

```

        key := arr[i]

        j := i - 1

        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }

        arr[j+1] = key
    }

    fmt.Println(arr)
}

```

The screenshot shows a Go IDE with a dark theme. On the left, the source code for a Go program is displayed, implementing an insertion sort algorithm. The code includes package declarations, imports, and a main function that sorts an array of integers. On the right, a preview window shows the output of the program, which consists of three lines of text: a long numeric string, a date-like string, and a name with underlined parts.

```

1  package main
2
3  import "fmt"
4
5  func main() {
6      arr := []int{5, 2, 9, 1, 5, 6}
7
8      for i := 1; i < len(arr); i++ {
9          key := arr[i]
10         j := i - 1
11
12         for j >= 0 && arr[j] > key {
13             arr[j+1] = arr[j]
14             j--
15         }
16
17         arr[j+1] = key
18     }
19
20     fmt.Println(arr)
21 }
22

```

Output:

```

109082500156
IF-13-02
Andromeda Alike Ramadhani

```

Output:

```

PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14\ContohSoal2\guided2.go"
[1 2 5 5 6 9]
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14>

```

Deskripsi Program:

Program diatas digunakan untuk mengurutkan sekumpulan data bilangan menggunakan algoritma Insertion Sort secara ascending. Pada awal program, dilakukan deklarasi sebuah slice bernama arr yang berisi beberapa bilangan integer, yaitu 5, 2, 9, 1, 5, dan 6. Data tersebut menjadi input awal yang akan diproses dan diurutkan oleh

program. Selain itu, program mengimpor package `fmt` yang digunakan untuk menampilkan hasil akhir pengurutan ke layar.

Proses utama pengurutan dilakukan menggunakan algoritma Insertion Sort, yaitu metode pengurutan yang bekerja dengan membandingkan dan menyisipkan elemen pada posisi yang sesuai di bagian data yang telah terurut. Perulangan dimulai dari indeks ke-1 karena elemen pertama dianggap sudah berada pada posisi yang benar. Pada setiap iterasi, nilai elemen yang sedang diproses disimpan ke dalam variabel `key`, sedangkan variabel `j` digunakan untuk menunjuk elemen sebelumnya yang akan dibandingkan.

Selanjutnya, program melakukan perulangan selama indeks `j` masih berada dalam batas array dan nilai pada posisi `arr[j]` lebih besar daripada `key`. Jika kondisi tersebut terpenuhi, elemen pada indeks `j` akan digeser satu posisi ke kanan untuk memberi ruang pada `key`. Proses ini dilakukan secara berulang hingga ditemukan posisi yang tepat, yaitu ketika elemen sebelumnya lebih kecil atau sama dengan `key`, atau ketika indeks sudah mencapai batas awal array. Setelah posisi yang sesuai ditemukan, nilai `key` ditempatkan pada indeks `j+1`.

Pada bagian akhir program, fungsi `fmt.Println(arr)` digunakan untuk menampilkan isi array yang telah selesai diurutkan. Hasil output menunjukkan bahwa seluruh data telah tersusun dari nilai terkecil ke terbesar.

SOAL LATIHAN

Statement Selection Sort

1. Soal1

Source Code:

```
package main

import "fmt"

const NMAX int = 1000

type tabInt [NMAX]int

func urutanRumah(arr *tabInt, n int) {
    var i, j, idxmin, temp int
    for i = 0; i < n-1; i++ {
        idxmin = i
        for j = i + 1; j < n; j++ {
            if arr[j] < arr[idxmin] {
                idxmin = j
            }
        }
        temp = arr[i]
        arr[i] = arr[idxmin]
        arr[idxmin] = temp
    }
}

func printData(arr tabInt, n int) {
    var i int
    for i = 0; i < n; i++ {
        fmt.Print(arr[i])
    }
}
```



```
        if i < n-1 {
            fmt.Print(" ")
        }
    }
    fmt.Print()
}

func main() {
    var nDaerah, m, i, j int
    var rumah tabInt

    fmt.Scan(&nDaerah)

    for i = 0; i < nDaerah; i++ {
        fmt.Scan(&m)
        for j = 0; j < m; j++ {
            fmt.Scan(&rumah[j])
        }
        urutanRumah(&rumah, m)
        printData(rumah, m)
    }
}
```

```
Soal1 > go rumahKerabat.go > main
1 package main
2
3 import "fmt"
4
5 const NMAX int = 1000
6
7 type tabInt [NMAX]int
8
9 func urutanRumah(arr *tabInt, n int) {
10     var i, j, idxmin, temp int
11     for i = 0; i < n-1; i++ {
12         idxmin = i
13         for j = i + 1; j < n; j++ {
14             if arr[j] < arr[idxmin] {
15                 idxmin = j
16             }
17         }
18         temp = arr[i]
19         arr[i] = arr[idxmin]
20         arr[idxmin] = temp
21     }
22 }
23
24 func printData(arr tabInt, n int) {
25     var i int
26     for i = 0; i < n; i++ {
27         fmt.Print(arr[i])
28         if i < n-1 {
29             fmt.Print(" ")
30         }
31     }
32 }
```

10908: 10908

File Edit View H1 ... 100% Windows (CI) UTF-8

109082500156
IF-13-02
Andromeda Alika Ramadhani

Ln 3, Col 26 | 47 characters | AA Forma... | 100% | Windows (CI) | UTF-8

Ln 44, Col 32 Tab Size: 4 UTF-8 CRLF Go 1.25.1 Go Live

Output:

```
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14\Soal1\rumahKerabat.go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14> 
```

Deskripsi Program:

Program tersebut dibuat untuk mengurutkan data nomor rumah di beberapa daerah secara ascending. Pada bagian awal program, didefinisikan konstanta NMAX bernilai 1000 yang digunakan sebagai batas maksimum jumlah data yang dapat disimpan. Selanjutnya, dibuat tipe data tabInt, yaitu array bertipe integer dengan kapasitas sebanyak NMAX. Tipe data ini digunakan untuk menyimpan kumpulan nomor rumah yang akan diproses.

Program memiliki fungsi urutanRumah yang berfungsi untuk mengurutkan data rumah dari nilai terkecil ke terbesar. Fungsi ini menggunakan algoritma *selection sort*, yaitu metode pengurutan dengan cara mencari elemen terkecil pada bagian array yang belum terurut, kemudian menukarnya dengan elemen pada posisi saat ini. Proses dilakukan menggunakan dua perulangan, di mana variabel idxmin digunakan untuk menyimpan indeks nilai terkecil, sedangkan variabel temp dipakai sebagai tempat sementara saat proses pertukaran data dilakukan.

Selain itu, terdapat fungsi printData yang bertugas menampilkan isi array setelah diurutkan. Fungsi ini melakukan perulangan untuk mencetak seluruh elemen array sesuai jumlah data yang dimasukkan. Setiap angka dipisahkan dengan spasi agar hasil keluaran lebih rapi dan mudah dibaca. Kondisi `if i < n-1` digunakan untuk mencegah adanya spasi tambahan di bagian akhir output.

Pada fungsi main, program dimulai dengan membaca jumlah daerah yang akan diproses melalui variabel `nDaerah`. Setelah itu, program melakukan perulangan sebanyak jumlah daerah tersebut. Untuk setiap daerah, pengguna diminta memasukkan jumlah rumah (`m`) yang akan diinput, kemudian seluruh nomor rumah disimpan ke dalam array `rumah`. Setelah data selesai dimasukkan, fungsi `urutanRumah` dipanggil untuk mengurutkan nomor rumah, lalu hasil pengurutan ditampilkan menggunakan fungsi `printData`.

2. Soal2

Source Code:

```
package main

import "fmt"

const NMAX int = 1000

type tabInt [NMAX]int

func urutGanjil(arr *tabInt, n int) {
    var i, j, idxmin, temp int
    for i = 0; i < n-1; i++ {
        idxmin = i
        for j = i + 1; j < n; j++ {
            if arr[j] < arr[idxmin] {
                idxmin = j
            }
        }
        temp = arr[i]
        arr[i] = arr[idxmin]
        arr[idxmin] = temp
    }
}

func urutGenap(arr *tabInt, n int) {
```

```

var i, j, idxmin, temp int
for i = 0; i < n-1; i++ {
    idxmin = i
    for j = i + 1; j < n; j++ {
        if arr[j] > arr[idxmin] {
            idxmin = j
        }
    }
    temp = arr[i]
    arr[i] = arr[idxmin]
    arr[idxmin] = temp
}

func printData(ganjil tabInt, nGanjil int, genap
tabInt, nGenap int) {
    var pertama bool
    var i int
    pertama = true
    for i = 0; i < nGanjil; i++ {
        if !pertama {
            fmt.Print(" ")
        }
        fmt.Print(ganjil[i])
        pertama = false
    }

    for i = 0; i < nGenap; i++ {
        if !pertama {
            fmt.Print(" ")
        }
    }
}

```

```

        fmt.Print(genap[i])

        pertama = false

    }

}

func main() {

    var nDaerah, nGanjil, nGenap, m, x, i, j int
    var ganjil, genap tabInt

    fmt.Scan(&nDaerah)

    for i = 0; i < nDaerah; i++ {

        nGanjil = 0
        nGenap = 0

        fmt.Scan(&m)

        for j = 0; j < m; j++ {

            fmt.Scan(&x)

            if x%2 == 1 {

                ganjil[nGanjil] = x
                nGanjil++

            } else {

                genap[nGenap] = x
                nGenap++

            }

        }

        urutGanjil(&ganjil, nGanjil)
        urutGenap(&genap, nGenap)
        printData(ganjil, nGanjil, genap, nGenap)

    }

}

```

```
Soal2 > go run rumahKerabat2.go ...
1 package main
2
3 import "fmt"
4
5 const NMAX int = 1000
6
7 type tabInt [NMAX]int
8
9 func urutGanjil(arr *tabInt, n int) {
10     var i, j, idxmin, temp int
11     for i = 0; i < n-1; i++ {
12         idxmin = i
13         for j = i + 1; j < n; j++ {
14             if arr[j] < arr[idxmin] {
15                 idxmin = j
16             }
17         }
18         temp = arr[i]
19         arr[i] = arr[idxmin]
20         arr[idxmin] = temp
21     }
22 }
23
24 func urutGenap(arr *tabInt, n int) {
25     var i, j, idxmin, temp int
26     for i = 0; i < n-1; i++ {
27         idxmin = i
28         for j = i + 1; j < n; j++ {
29             if arr[j] > arr[idxmin] {
30                 idxmin = j
31             }
32         }
33         temp = arr[i]
34         arr[i] = arr[idxmin]
35         arr[idxmin] = temp
36     }
37 }
38
39 func printData(arr *tabInt, n int) {
40     for i = 0; i < n; i++ {
41         fmt.Print(arr[i])
42     }
43     fmt.Println()
44 }
45
46 func main() {
47     arr := tabInt{}
48     n := 0
49     for i = 0; i < 1000; i++ {
50         if i%2 == 0 {
51             arr[i] = i
52         } else {
53             arr[i] = i+1
54         }
55         n++
56     }
57     urutGanjil(&arr, n)
58     urutGenap(&arr, n)
59     printData(&arr, n)
60 }
```

```
109082500156
IF-13-02
Andromeda Alika Ramadhani
```

Ln 3, Col 26 | 47 characters | A1 Forma... | 100% | Windows (CI) | UTF-8

Ln 83, Col 1 | Tab Size: 4 | UTF-8 | CRLF | Go | 1.25.1 | Go Live

Output:

```
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14\Soal2\rumahKerabat2.go"
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14>
```

Deskripsi Program:

Program diatas digunakan untuk mengelompokkan dan mengurutkan nomor rumah berdasarkan jenis bilangan, yaitu bilangan ganjil dan bilangan genap. Pada bagian awal program, didefinisikan konstanta NMAX dengan nilai 1000 sebagai batas maksimum kapasitas array yang digunakan untuk menyimpan data. Selain itu, dibuat tipe data tabInt, yaitu array bertipe integer yang berfungsi untuk menyimpan kumpulan bilangan yang akan diproses selama program berjalan.

Program memiliki dua fungsi pengurutan, yaitu urutGanjil dan urutGenap. Fungsi urutGanjil digunakan untuk mengurutkan bilangan ganjil secara ascending dari nilai terkecil ke terbesar menggunakan algoritma selection sort. Cara kerja algoritma ini adalah dengan mencari nilai terkecil dari bagian array yang belum terurut, kemudian menukarnya dengan elemen pada posisi saat ini. Sementara itu, fungsi urutGenap digunakan untuk mengurutkan bilangan genap secara descending dari nilai terbesar ke terkecil. Proses pengurutan pada fungsi ini juga menggunakan metode selection sort, namun kondisi pembandingnya dibalik sehingga nilai terbesar dipilih terlebih dahulu.

Selanjutnya, terdapat fungsi printData yang berfungsi untuk menampilkan hasil akhir data setelah dipisahkan dan diurutkan. Fungsi ini terlebih dahulu mencetak semua bilangan ganjil yang telah diurutkan, kemudian dilanjutkan dengan bilangan genap.

Variabel pertama bertipe boolean digunakan untuk mengatur format output agar setiap angka dipisahkan oleh spasi tanpa menghasilkan spasi berlebih di awal maupun akhir tampilan.

Pada fungsi main, program diawali dengan membaca jumlah daerah yang akan diproses melalui variabel `nDaerah`. Untuk setiap daerah, program menerima input jumlah data rumah (`m`) dan membaca setiap bilangan satu per satu melalui variabel `x`. Data kemudian dipisahkan berdasarkan jenis bilangan, di mana bilangan ganjil disimpan ke dalam array ganjil, sedangkan bilangan genap disimpan ke dalam array genap. Setelah proses pemisahan selesai, data ganjil diurutkan menggunakan fungsi `urutGanjil`, sedangkan data genap diurutkan menggunakan fungsi `urutGenap`. Hasil akhirnya ditampilkan melalui fungsi `printData`, sehingga output program menampilkan seluruh bilangan ganjil dalam urutan menaik terlebih dahulu, kemudian diikuti bilangan genap dalam urutan menurun.

3. Soal3

Source Code:

```
package main

import "fmt"

const NMAX int = 1000000

type tabInt [NMAX]int

func pengurutanData(arr *tabInt, n int) {
    var i, j, key int
    for i = 1; i < n; i++ {
        key = arr[i]
        j = i - 1
        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }
        arr[j+1] = key
    }
}
```

```

func median(arr tabInt, n int) int {
    var tengah int
    if n%2 == 1 {
        tengah = n / 2
        return arr[tengah]
    }
    tengah = n / 2
    return (arr[tengah-1] + arr[tengah]) / 2
}

func main() {
    var data tabInt
    var n, a int
    n = 0
    for {
        fmt.Scan(&a)
        if a == -5313 {
            break
        }
        if a == 0 {
            pengurutanData(&data, n)
            fmt.Println(median(data, n))
        } else {
            data[n] = a
            n++
        }
    }
}

```



```
Soal3 > median.go > pengurutanData
1 package main
2
3 import "fmt"
4
5 const NMAX int = 1000000
6
7 type tabInt [NMAX]int
8
9 func pengurutanData(arr *tabInt, n int) {
10     var i, j, key int
11     for i = 1; i < n; i++ {
12         key = arr[i]
13         j = i - 1
14         for j >= 0 && arr[j] > key {
15             arr[j+1] = arr[j]
16             j--
17         }
18         arr[j+1] = key
19     }
20 }
21
22 func median(arr tabInt, n int) int {
23     var tengah int
24     if n%2 == 1 {
25         tengah = n / 2
26         return arr[tengah]
27     }
28     tengah = n / 2
29     return (arr[tengah-1] + arr[tengah]) / 2
30 }
```

10908: •

File Edit View H1 ≡ ⋮ ↻ 🔍 👤 ⚙

109082500156
IF-13-02
Andromeda Alika Ramadhani

Ln 3, Col 26 | 47 characters | AA Forma... | 100% | Windows (CI) | UTF-8

Ln 18, Col 16 | Tab Size: 4 | UTF-8 | CRLF | Go 1.25.1 | Go Live

Output:

```
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14\Soal3\median.go"
7 23 11 0 5 19 2 29 3 13 17 0 -5313
11
12
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14> 
```

Deskripsi Program:

Program diatas digunakan untuk mencari nilai median dari sekumpulan data ketika ditemukan input tertentu. Pada bagian awal program, didefinisikan konstanta NMAX bernilai 1.000.000 yang berfungsi sebagai batas maksimum jumlah data yang dapat disimpan. Selain itu, dibuat tipe data tabInt, yaitu array bertipe integer yang digunakan untuk menampung seluruh data bilangan yang dimasukkan pengguna selama program berjalan.

Program memiliki fungsi pengurutanData yang digunakan untuk mengurutkan data menggunakan algoritma insertion sort. Algoritma ini bekerja dengan membandingkan setiap elemen dengan elemen-elemen sebelumnya, kemudian menempatkannya pada posisi yang sesuai agar data tersusun secara ascending. Variabel key digunakan untuk menyimpan nilai yang sedang diproses, sedangkan variabel j digunakan untuk menelusuri posisi sebelumnya pada array. Jika ditemukan nilai yang lebih besar dari key, maka elemen tersebut akan digeser ke kanan hingga posisi yang tepat ditemukan.

Selanjutnya, terdapat fungsi median yang berfungsi untuk menghitung nilai median dari data yang telah diurutkan. Median merupakan nilai tengah dari sekumpulan data. Jika jumlah data bersifat ganjil, maka median diambil langsung dari elemen yang berada di posisi tengah array. Namun, jika jumlah data genap, median diperoleh dari

hasil rata-rata dua nilai tengah. Posisi tengah ditentukan menggunakan variabel tengah, yang diperoleh dari pembagian jumlah data dengan dua.

Pada fungsi main, program dimulai dengan mendeklarasikan array data sebagai tempat penyimpanan bilangan yang dimasukkan pengguna. Variabel n digunakan untuk menghitung jumlah data yang tersimpan, sedangkan variabel a digunakan untuk membaca input. Program berjalan menggunakan infinite loop yang akan terus menerima input sampai ditemukan nilai -5313, yang berfungsi sebagai tanda akhir program. Jika input yang diterima adalah 0, maka program akan mengurutkan seluruh data yang sudah tersimpan menggunakan fungsi pengurutanData, lalu menghitung dan menampilkan median melalui fungsi median. Namun, jika input bukan 0 maupun -5313, nilai tersebut akan disimpan ke dalam array data dan jumlah data akan bertambah satu.

SOAL LATIHAN

Statement Insertion Sort

1. Soal1

Source Code:

```
package main

import "fmt"

const NMAX int = 1000

type tabInt [NMAX]int

func pengurutanData(arr *tabInt, n int) {
    var i, j, key int
    for i = 1; i < n; i++ {
        key = arr[i]
        j = i - 1
        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }
        arr[j+1] = key
    }
}
```

```

    }

}

func cekJarak(arr tabInt, n int) (bool, int) {
    var jarak, i int
    var jarakTetap bool
    jarakTetap = true
    if n < 2 {
        return true, 0
    }
    jarak = arr[1] - arr[0]
    for i = 2; i < n; i++ {
        if arr[i]-arr[i-1] != jarak {
            jarakTetap = false
        }
    }
    return jarakTetap, jarak
}

func main() {
    var data tabInt
    var jarak, n, x, i int
    var jarakTetap bool
    n = 0
    for {
        fmt.Scan(&x)
        if x < 0 {
            break
        }
        data[n] = x
        n++
    }
}

```

```

    }

    pengurutanData(&data, n)

    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    fmt.Println()

    jarakTetap, jarak = cekJarak(data, n)

    if jarakTetap {
        fmt.Println("Data berjarak ", jarak)
    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
}

```

The screenshot shows a Go IDE with two panels. The left panel displays the source code for a program named 'jarakData.go'. The code includes a package declaration, imports, constants, type definitions, and two functions: 'pengurutanData' (a bubble sort implementation) and 'cekJarak' (a function to check if an array is an arithmetic progression). The right panel shows the output of the program, which is displayed in a terminal window. The output consists of three lines: a long number '109082500156', a string 'IF-13-02', and a name 'Andromeda Alika Ramadhani' where 'Alika' and 'Ramadhani' are underlined.

```

Soal4 > -o jarakData.go > pengurutanData
1  package main
2
3  import "fmt"
4
5  const NMAX int = 1000
6
7  type tabInt [NMAX]int
8
9  func pengurutanData(arr *tabInt, n int) {
10     var i, j, key int
11     for i = 1; i < n; i++ {
12         key = arr[i]
13         j = i - 1
14         for j >= 0 && arr[j] > key {
15             arr[j+1] = arr[j]
16             j--
17         }
18         arr[j+1] = key
19     }
20 }
21
22 func cekJarak(arr tabInt, n int) (bool, int) {
23     var jarak, i int
24     var jarakTetap bool
25     jarakTetap = true
26     if n < 2 {
27         return true, 0
28     }
29     jarak = arr[1] - arr[0]

```

109082500156
IF-13-02
Andromeda Alika Ramadhani

Output:

```

PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14\Soal14\jarakData.go"
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14\Soal14\jarakData.go"
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14>

```

Deskripsi Program:

Program diatas digunakan untuk memeriksa jarak tetap antar elemen data. Pada bagian awal program, didefinisikan konstanta NMAX dengan nilai 1000 yang berfungsi sebagai batas maksimum kapasitas penyimpanan data. Selanjutnya, dibuat tipe data tabInt, yaitu array bertipe integer yang digunakan untuk menyimpan bilangan-bilangan yang dimasukkan oleh pengguna.

Program memiliki fungsi pengurutanData yang bertugas mengurutkan data menggunakan algoritma insertion sort. Metode ini bekerja dengan mengambil satu elemen, kemudian membandingkannya dengan elemen-elemen sebelumnya untuk ditempatkan pada posisi yang sesuai sehingga data tersusun secara ascending. Pada fungsi ini, variabel key digunakan untuk menyimpan nilai yang sedang diproses, sedangkan variabel j berfungsi untuk menelusuri elemen sebelumnya dan menggeser data jika ditemukan nilai yang lebih besar dari key.

Selain fungsi pengurutan, terdapat fungsi cekJarak yang digunakan untuk memeriksa apakah data yang telah diurutkan memiliki selisih yang sama antar elemen secara berurutan. Fungsi ini menghasilkan dua nilai keluaran, yaitu sebuah nilai boolean (true atau false) untuk menunjukkan apakah jaraknya tetap, serta nilai integer yang menunjukkan besar selisih tersebut. Jika jumlah data kurang dari dua, fungsi langsung mengembalikan nilai true dan jarak 0 karena belum dapat dilakukan perbandingan. Apabila data berjumlah dua atau lebih, program terlebih dahulu menentukan jarak awal berdasarkan selisih dua elemen pertama, kemudian membandingkannya dengan selisih antar elemen berikutnya. Jika ditemukan selisih yang berbeda, maka variabel jarakTetap akan berubah menjadi false, yang menandakan bahwa data tidak memiliki pola jarak yang konsisten.

Pada fungsi main, program dimulai dengan membaca input bilangan dari pengguna secara berulang menggunakan infinite loop. Input akan terus diterima hingga pengguna memasukkan bilangan negatif, yang digunakan sebagai tanda penghentian proses input. Setiap bilangan non-negatif disimpan ke dalam array data, sementara variabel n digunakan untuk menghitung jumlah data yang telah dimasukkan. Setelah proses input selesai, data akan diurutkan menggunakan fungsi pengurutanData, kemudian seluruh data hasil pengurutan ditampilkan ke layar.

Setelah data berhasil diurutkan, program memanggil fungsi cekJarak untuk menentukan apakah data memiliki selisih yang tetap. Jika hasil pemeriksaan menunjukkan bahwa jaraknya konsisten, program akan menampilkan informasi mengenai besar jarak antar data tersebut. Namun, jika terdapat perbedaan selisih antar elemen, program akan menampilkan keterangan bahwa data memiliki jarak yang tidak tetap.

2. Soal2

Source Code:

```
package main

import "fmt"

const nMax int = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    ekslempar, tahun, rating      int
}

type DaftarBuku [nMax]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
    var i int
    fmt.Scan(n)
    for i = 0; i < *n; i++ {
        fmt.Scan(&pustaka[i].id, &pustaka[i].judul,
            &pustaka[i].penulis, &pustaka[i].penerbit,
            &pustaka[i].ekslempar, &pustaka[i].tahun,
            &pustaka[i].rating)
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    var idxfav, i int
    idxfav = 0
    for i = 0; i < n; i++ {
        if pustaka[i].rating > pustaka[idxfav].rating {
            idxfav = i
        }
    }
}
```

```

    }

    }

    fmt.Printf("%s, %s, %s, %d\n",
        pustaka[idxfav].judul, pustaka[idxfav].penulis,
        pustaka[idxfav].penerbit, pustaka[idxfav].tahun)
}

func UrutBuku(pustaka *DaftarBuku, n *int) {
    var i, j int
    var key Buku
    for i = 1; i < *n; i++ {
        key = pustaka[i]
        j = i - 1
        for j >= 0 && pustaka[j].rating < key.rating {
            pustaka[j+1] = pustaka[j]
            j--
        }
        pustaka[j+1] = key
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    var batas, i int
    batas = 5
    if n < 5 {
        batas = n
    }
    for i = 0; i < batas; i++ {
        fmt.Print(pustaka[i].judul, " ")
    }
    fmt.Println()
}

```

```

    }

    func CariBuku(pustaka DaftarBuku, n, r int) {
        var kiri, kanan, ketemu, median int
        var b Buku
        kiri = 0
        kanan = n - 1
        ketemu = -1
        for kiri <= kanan && ketemu == -1 {
            median = (kiri + kanan) / 2
            if pustaka[median].rating == r {
                ketemu = median
            } else if pustaka[median].rating < r {
                kanan = median - 1
            } else {
                kiri = median + 1
            }
        }
        if ketemu != -1 {
            b = pustaka[ketemu]
            fmt.Printf("%s %s %s %d %d %d\n", b.judul,
                b.penulis, b.penerbit,
                b.tahun, b.ekslempar, b.rating)
        } else {
            fmt.Println("Tidak ada buku dengan rating
                seperti itu")
        }
    }

    func main() {
        var pustaka DaftarBuku
    }

```



```

    var nPustaka, ratingDicari int

    DaftarkanBuku(&pustaka, &nPustaka)

    fmt.Print("Masukkan rating buku yang dicari: ")

    fmt.Scan(&ratingDicari)

    CetakTerfavorit(pustaka, nPustaka)

    UrutBuku(&pustaka, &nPustaka)

    Cetak5Terbaru(pustaka, nPustaka)

    CariBuku(pustaka, nPustaka, ratingDicari)

}

```

The screenshot shows a Go IDE with a source code editor on the left and a terminal/output window on the right. The source code defines a program for managing a book collection. The terminal window shows the output of the program, which is a list of book details: ID, IF number, author, publisher, and rating.

```

Soal5 > cd perpustakaan.go > DaftarkanBuku
1 package main
2
3 import "fmt"
4
5 const nMax int = 7919
6
7 type Buku struct {
8     id, judul, penulis, penerbit string
9     ekslempar, tahun, rating    int
10 }
11
12 type DaftarBuku [nMax]Buku
13
14 func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
15     var i int
16     fmt.Scan(n)
17     for i = 0; i < *n; i++ {
18         fmt.Scan(&pustaka[i].id, &pustaka[i].judul, &pustaka[i].penulis, &pustaka[i].penerbit,
19             &pustaka[i].ekslempar, &pustaka[i].tahun, &pustaka[i].rating)
20     }
21 }
22
23 func CetakTerfavorit(pustaka DaftarBuku, n int) {
24     var idxfav, i int
25     idxfav = 0
26     for i = 0; i < n; i++ {
27         if pustaka[i].rating > pustaka[idxfav].rating {
28             idxfav = i
29         }
30     }
31 }

```

Terminal Output:

```

109082500156
IF-13-02
Andromeda Alika Ramadhani

```

Ln 3, Col 26 | 47 characters | AA Forma... | 100% | Windows (C) | UTF-8

Ln 16, Col 16 | Tab Size: 4 | UTF-8 | CRLF | Go | 1.25.1 | Go Live

Output:

```
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14\Soal5\perpustakaan.go"
3
B01 Laskar_Pelangi Andrea_Hirata Bentang_Pustaka 10 2005 9
B02 Bumi_Manusia Pramoedya_Ananta_Toer Lentera_Dipantara 5 1980 10
B03 Negeri_5_Menara A.Fuadi Gramedia 7 2009 8
Masukkan rating buku yang dicari: 9
Bumi_Manusia, Pramoedya_Ananta_Toer, Lentera_Dipantara, 1980
Bumi_Manusia Laskar_Pelangi Negeri_5_Menara
Laskar_Pelangi Andrea_Hirata Bentang_Pustaka 2005 10 9
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL14>
```

Deskripsi Program:

Program diatas digunakan untuk mengelola data perpustakaan, mulai dari proses pendataan buku, pencarian buku favorit, pengurutan berdasarkan rating, menampilkan beberapa buku dengan rating tertinggi, hingga mencari buku berdasarkan nilai rating tertentu. Pada bagian awal program, didefinisikan konstanta `nMax` bernilai 7919 yang berfungsi sebagai batas maksimum jumlah data buku yang dapat disimpan. Selain itu, dibuat sebuah struct bernama `Buku` yang digunakan untuk merepresentasikan informasi setiap buku. Struktur ini terdiri dari id, judul, penulis, dan penerbit bertipe string, serta ekslemplar, tahun, dan rating bertipe integer. Selanjutnya, dibuat tipe data `DaftarBuku`, yaitu array bertipe `Buku` yang digunakan sebagai tempat penyimpanan seluruh data pustaka.

Program memiliki fungsi `DaftarkanBuku` yang berfungsi untuk menerima input data buku dari pengguna. Pada fungsi ini, pengguna terlebih dahulu memasukkan jumlah buku yang akan didaftarkan, kemudian program melakukan perulangan untuk membaca seluruh atribut buku, seperti identitas buku, judul, nama penulis, penerbit, jumlah eksemplar, tahun terbit, dan nilai rating. Seluruh data tersebut disimpan ke dalam array pustaka agar dapat diproses lebih lanjut.

Fungsi `CetakTerfavorit` digunakan untuk menentukan buku dengan rating tertinggi. Program melakukan pencarian dengan membandingkan rating setiap buku menggunakan perulangan. Variabel `idxfav` digunakan untuk menyimpan indeks buku dengan rating terbesar. Setelah seluruh data diperiksa, program menampilkan informasi buku favorit berupa judul, nama penulis, penerbit, dan tahun terbit. Dengan demikian, fungsi ini bertujuan untuk mengetahui buku yang paling diminati berdasarkan penilaian rating.

Selanjutnya, fungsi `UrutBuku` digunakan untuk mengurutkan data buku berdasarkan rating secara descending menggunakan algoritma insertion sort. Pada proses ini, buku dengan rating lebih tinggi akan ditempatkan pada posisi awal array. Variabel `key` bertipe `Buku` digunakan untuk menyimpan data sementara saat proses penggeseran elemen berlangsung. Dengan pengurutan ini, data buku menjadi lebih terstruktur sehingga mempermudah proses pencarian maupun penampilan daftar buku terbaik.

Setelah data berhasil diurutkan, fungsi `Cetak5Terbaru` digunakan untuk menampilkan maksimal lima judul buku pertama dari hasil pengurutan. Karena data telah diurutkan berdasarkan rating tertinggi, maka buku yang ditampilkan merupakan lima buku dengan rating terbaik. Namun, apabila jumlah buku kurang dari lima, program hanya akan menampilkan seluruh buku yang tersedia. Hasil output ditampilkan dalam satu baris dengan pemisah spasi antar judul buku.

Program juga memiliki fungsi CariBuku yang digunakan untuk mencari buku berdasarkan rating tertentu menggunakan algoritma binary search. Metode ini bekerja dengan membagi data menjadi dua bagian dan mencari nilai target di posisi tengah. Karena data sebelumnya sudah diurutkan berdasarkan rating menurun, proses pencarian dapat dilakukan lebih cepat dibanding pencarian linear biasa. Jika ditemukan buku dengan rating yang sesuai, program akan menampilkan detail informasi buku tersebut, meliputi judul, penulis, penerbit, tahun terbit, jumlah eksemplar, dan rating. Namun, jika tidak ditemukan data dengan rating yang dicari, program akan menampilkan pesan bahwa buku dengan rating tersebut tidak tersedia.

Pada fungsi main, program dimulai dengan mendeklarasikan variabel pustaka untuk menyimpan data buku, nPustaka untuk jumlah data buku, dan ratingDicari untuk menyimpan rating yang ingin dicari pengguna. Program terlebih dahulu memanggil fungsi DaftarkanBuku untuk memasukkan data, kemudian meminta pengguna menginput rating buku yang ingin dicari. Setelah itu, program menampilkan buku terfavorit melalui fungsi CetakTerfavorit, mengurutkan data dengan fungsi UrutBuku, menampilkan lima buku terbaik menggunakan fungsi Cetak5Terbaru, dan terakhir mencari buku berdasarkan rating melalui fungsi CariBuku.

DAFTAR PUSTAKA

Program Studi S1 Informatika. (2026). *Modul algoritma dan pemrograman 2 IF REG 2026*. Universitas Telkom.

Sekretariat Kampus Daerah Purwokerto. (2023, Desember 22). *Berbagai macam algoritma sort data*. Telkom University Kampus Purwokerto. <https://bif-pwt.telkomuniversity.ac.id/berbagai-macam-algoritma-sort-data/>

GeeksforGeeks. (2024, November 8). *Insertion sort algorithm*. GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/insertion-sort-algorithm/>

GeeksforGeeks. (2024, November 11). *Selection sort algorithm*. GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/selection-sort-algorithm-2/>